# HoldSpec: A Machine-Checked Model and Conformance Suite for the Payment Authorization Hold Lifecycle

Abhishek Sharma
Senior Member, IEEE
abhicse24@gmail.com | github.com/abhisheksharma2411 | abhisharma.co.in

***Abstract*—When a merchant authorizes a card and captures the funds later, the money sits in a state that neither party fully controls: held on the cardholder's account, not yet taken, and due to lapse on a deadline set by the card network. What a merchant may do to that hold — capture part of it, capture again, void it, raise it, let it expire — is business logic that every payment service provider implements slightly differently and none of them specifies formally. We give the lifecycle a state machine in TLA+, parameterized by a provider capability profile, and check eight safety invariants and three liveness properties over 6 profiles, five of them built from Stripe's and Adyen's published documentation. From the same machine we generate a conformance suite that runs black-box against a provider API. Against 61 seeded defects that are distinguishable from a correct implementation, the generated suite detects 100%, where random sequences with the same call budget reach 56% and a hand-written integration suite reaches 69%; random sequences given ten times the budget still reach only 67%. Running the same suite between two providers turns the comparison into a search, and it returns short executable witnesses for divergences the two documentation sets imply but neither states — including one that breaks the common assumption that a single adapter interface can sit over both, because closing a hold and releasing its remainder is a capture on one provider and a cancel on the other. We report what we could not do: no provider sandbox credentials were available, so every result here concerns documented behavior, and the suite that would test the providers themselves is part of what we release.**

***Index Terms*—formal specification, TLA+, model checking, conformance testing, model-based testing, payment systems, authorization and capture**

## I. Introduction

A hotel takes a card at check-in and charges it at check-out. A retailer authorizes an order and captures each parcel as it ships. A car rental firm holds an estimate and settles the real amount days later. All three use the same mechanism: an authorization that reserves funds without moving them, followed by one or more captures that move some or all of what was reserved, and a deadline after which whatever was not captured goes back.

The mechanism is old and the rules around it are not written down anywhere a machine can read. Each payment service provider (PSP) documents its own version in prose: how long an authorization stays good, whether a second capture is possible, whether more than the authorized amount may be taken, what happens to the remainder after a partial capture. The rules differ between providers, they differ between accounts at the same provider, and the differences are the kind that only surface in production, on somebody's money.

The defects are correspondingly unglamorous and expensive. A capture that arrives after a void. A retry loop that captures twice against one authorization. An over-capture that a provider silently accepts and a card network later charges back. A hold that closes without the funds being released, so a customer's balance stays short for a week. Each is a violation of a rule that a state machine could state in one line, and that no PSP states in a form anything can check.

This paper writes that state machine down and then puts it to work.

### A. Contributions

- **A formal model of the hold lifecycle** (§III). A TLA+ specification of authorization, partial and multiple capture, void, expiry, incremental authorization, and the release of held funds, parameterized by a provider capability profile. Eight safety invariants and three liveness properties are checked exhaustively over 6 profiles; 4,096 distinct states in total, in 4.0 seconds.
- **Evidence that the checking is not vacuous** (§IV). 10 targeted changes to the specification, one per property. Every property is falsified by the change aimed at it, and no change survives every property.
- **A conformance suite generated from the model** (§V), executable black-box over HTTP. On 61 seeded defects it detects 100%, against 56% for equal-budget random testing and 69% for a hand-written suite. Defects that no call sequence can distinguish from correct behavior are identified by exhaustive search and reported separately rather than counted as misses.
- **A differential procedure for comparing providers** (§VI-C) that enumerates, exhaustively within the model's bounds, every way two profiles disagree, with the shortest call sequence that shows each one. Applied to Stripe and Adyen it returns divergences in authorization validity, over-capture, and the mechanism for releasing a remainder.
- **Two negative results about observability** (§V-C) that we did not expect and that constrain any conformance work in this area: neither the number of captures nor the

reason a hold closed is a provider-neutral observable, and the second is not synchronously observable at all.

Everything is released under MIT: the specification, the model checker configurations, the suite, the mock providers, the sandbox adapters, and the scripts that regenerate every number and figure here.

## II. The hold lifecycle

### A. What a hold is

An authorization asks the issuer to reserve an amount on a card. The issuer answers yes or no, and on yes the amount stops being available to the cardholder without yet belonging to the merchant. Capture is the separate step that moves it. Between the two, three things can happen: the merchant captures (possibly less than was authorized, possibly more than once, possibly more than was authorized), the merchant voids, or the clock runs out.

The rules are not the merchant's to choose. Card networks set the validity window, and a capture after it may be rejected or may incur a fee. The provider's API surface then decides which of the network's possibilities the merchant can actually reach.

### B. Why this is not idempotency

Duplicate-effect prevention is the neighboring problem and a different one. Idempotency is about one operation submitted more than once; the hold lifecycle is about one authorization over its life. An implementation can be flawlessly idempotent and still accept a capture after a void, because the capture is not a retry of anything — it is a first attempt at an operation that should no longer be legal. It can be correct about holds and still double-charge on a network retry. The two are independent, and we treat them separately: the idempotency contract is the subject of a companion manuscript [1], and nothing here depends on it.

### C. What the providers document

Table I gives the profiles used throughout, each field taken from published documentation with the URL and the quotation recorded in the artifact. Four points are worth drawing out because the experiments turn on them.

Stripe permits one capture on most payments; a partial capture releases the remainder automatically [2]. With multicapture, up to 50 non-final captures are allowed followed by one final capture, and `final_capture=false` keeps the remaining amount authorized [3]. With overcapture, a capture may exceed the authorized amount by a network- and category-dependent margin, generally 15% and up to 30% for some merchant categories [4].

Adyen's default is a single partial capture after which "[a]ny unclaimed amount that is left over after partially capturing a payment is automatically cancelled"; enabling multiple partial captures changes that, so "[t]he unclaimed amount after an initial partial capture is not automatically cancelled" [5]. A capture amount "must be the same as or, in case of a partial capture, less than the authorized amount" — no over-capture. Adyen's validity table gives 10 days for Visa card-not-present cardholder-initiated transactions [6], where Stripe documents 7 days for the same brand [2].

Two structural differences follow from this and matter more than any single number. Stripe's API has a flag that says "this capture is the last one"; Adyen's has no such flag, so on an account with multiple partial captures the merchant ends a hold either by capturing the whole authorized amount or by cancelling what is left. And where Stripe releases a remainder with a zero-amount capture, Adyen releases it with a cancel.

## III. The model

### A. State and actions

A hold is described by eleven variables: its `state` (`NONE`/`HELD`/`CLOSED`), how it closed, the authorized amount, the captured total, the captured total recorded at the moment of closing, the number of accepted captures, the time of the last capture, a count of funds-release events, the expiry instant, the release deadline, and a clock. Amounts are integer minor units; time is in ticks.

Eight actions move it. `Authorize` creates the hold. `CaptureNonFinal` takes part of it and leaves the rest reserved. `CaptureFinal` takes an amount and closes the hold, releasing whatever is left; an amount of zero is the documented way to release a remainder without taking more. `Void` closes an open hold. `Expire` closes it at its deadline. `ReleaseHold` returns the reserved funds to the cardholder. `IncreaseAuth` raises the authorized amount. `Tick` advances the clock.

Keeping `ReleaseHold` separate from the actions that close a hold is what makes the interesting properties interesting. If closing a hold released its funds in the same step, "released exactly once" and "released promptly" would be true by construction and worth nothing. Split apart, they are claims about the implementation that a suite can actually test — and, as §V shows, the mutants that break them are exactly the ones a hand-written suite tends to miss.

### B. Profiles as parameters

Everything provider-specific is a constant: the authorized amount and its ceiling, the over-capture allowance, the non-final capture budget, whether incremental authorization is offered, whether a final-capture flag exists, whether a void is accepted after a capture, the validity window, and the release deadline. One specification therefore covers every profile in Table I, and a provider difference is a difference in constants rather than in code. This is what makes §VI-C possible.

### C. Time

Two deadlines behave differently, and the difference is deliberate. An authorization expires at its deadline: the clock cannot advance past `expiresAt` while a hold is open, so `Expire` fires there. Releasing the funds is allowed to lag, but only by the profile's release delay, and once that is exceeded the clock is blocked again. Both use the upper-bound-timer treatment of real time in TLA+ [7]: an action with a deadline

TABLE I
PROVIDER PROFILES, FROM PUBLISHED DOCUMENTATION (ACCESSED 1 SEPTEMBER 2026). THE SYNTHETIC PROFILE EXERCISES INCREMENTAL AUTHORIZATION, WHICH NEITHER PROVIDER'S DOCUMENTED DEFAULT INCLUDES.

| Profile | Validity (days) | Captures allowed | Over-capture | Final-capture flag | Void after partial capture | Remainder released by |
|---|---|---|---|---|---|---|
| Stripe, card default | 7 | 1 | 0% | yes | no | automatic on partial capture |
| Stripe, multicapture | 7 | 51 | 0% | yes | no | zero-amount capture with final_capture=true |
| Stripe, overcapture | 7 | 1 | 15% | yes | no | automatic on partial capture |
| Adyen, card default | 10 | 1 | 0% | yes | no | automatic on partial capture |
| Adyen, multiple partial captures | 10 | not fixed | 0% | no | yes | explicit cancel of the remaining amount |
| Incremental auth (synthetic) | 7 | 2 | 0% | yes | yes | explicit cancel of the remaining amount |

TABLE II
TLC RESULTS PER PROFILE. EIGHT INVARIANTS AND THREE LIVENESS PROPERTIES WERE ENABLED FOR EVERY RUN.

| Profile | States | Distinct | Depth | Time (s) | Verdict |
|---|---|---|---|---|---|
| Stripe, card default | 182 | 131 | 9 | 0.6 | pass |
| Stripe, multicapture | 1,032 | 642 | 11 | 0.7 | pass |
| Stripe, overcapture | 215 | 155 | 9 | 0.6 | pass |
| Adyen, card default | 249 | 182 | 10 | 0.6 | pass |
| Adyen, multiple partial captures | 1,067 | 738 | 12 | 0.7 | pass |
| Incremental auth (synthetic) | 3,509 | 2,248 | 11 | 0.8 | pass |

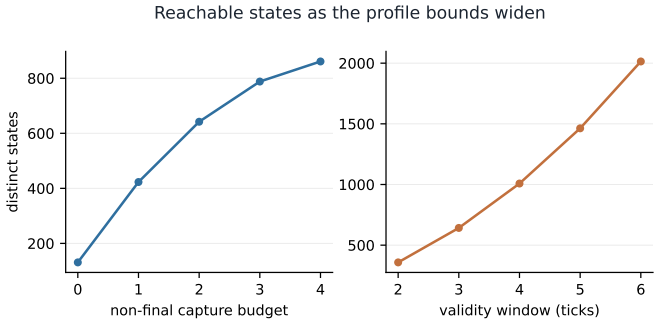


Fig. 1. The reachable space grows roughly linearly in the capture budget and faster in the validity window, since a longer window multiplies the states in which each capture can happen. Both stay small enough for exhaustive checking across the range a provider profile needs.

blocks time rather than being quietly missed, so a missed deadline becomes a timelock that the model checker reports.

This was not the first design. Initially the clock advanced freely and `INV_BoundedRelease` was left as an invariant to check. TLC's first run produced a four-step counterexample in which a hold closed, time simply ran on, and the release never became due — correctly reporting that the specification permitted what it was meant to forbid.

### D. Properties

Eight safety invariants: captures never exceed the profile's ceiling; nothing is captured after a hold closes; funds are released at most once; funds are not released before the hold closes; a closed hold releases within its deadline; no capture is accepted at or after the expiry instant; the capture-count budget is respected; and a released hold leaves nothing held. Three liveness properties, under weak fairness on `Authorize`, `Tick`, `Expire` and `ReleaseHold`: a closed hold eventually releases; an open hold does not stay open forever; every behavior reaches a settled end state.

### E. Model checking

Table II gives TLC's results. All eleven properties hold for all 6 profiles. The state spaces are small — the largest is 2,248 distinct states — which is the point of a bounded model: exhaustive checking finishes in 4.0 seconds in total, so the specification can be re-checked on every change to it, as Howard et al. [8] argue specifications should be.

## IV. IS THE CHECKING VACUOUS?

A specification that satisfies its invariants because nothing can happen in it is worthless, and the failure mode is quiet. To test for it we broke the specification 10 times, once per property, and re-ran TLC.

Each mutation removes or weakens one guard: a capture that ignores the authorization deadline, a capture that ignores the ceiling, a release that can happen twice, a release that can happen while the hold is open, a clock that runs past an overdue release, a capture budget that is not enforced, a close that forgets what had been captured, and the removal of each of the three fairness conditions.

Attribution needs care. TLC stops at the first property that fails, so with everything enabled a mutation can be reported against a property it was not aimed at — both ceiling mutations trip `TypeOK` first, and two of the three fairness mutations trip `EventualRelease` before their own property. Each mutation is therefore run twice: once with all properties, and once with only the property it targets. Table III reports the second. All 10 are caught by the property aimed at them, and 0 survive every property.

Two limits on what this shows. It establishes that each property has force, not that the set is complete: a rule we never wrote down cannot fail here. And the profile capability guards are not covered, because they are not safety properties — weakening one makes the model describe a different provider rather than an incorrect one. Measuring those is what §VI-C does.

TABLE III
SPECIFICATION MUTATIONS. "CAUGHT" IS WITH ONLY THE TARGETED PROPERTY ENABLED. TRACE IS THE COUNTEREXAMPLE LENGTH TLC REPORTED.

| | Change made to the specification | Property targeted | Caught | Trace |
|---|---|---|---|---|
| S01 | a final capture no longer checks the authorization deadline | `NoCaptureAfterExpiry` | yes | 7 |
| S02 | a final capture no longer checks the profile's capture ceiling | `CaptureWithinLimit` | yes | 5 |
| S03 | the hold may be released again after it has already been released | `ReleaseAtMostOnce` | yes | 7 |
| S04 | held funds may be released before the hold closes | `NoReleaseBeforeClose` | yes | 5 |
| S05 | time may pass while a release is overdue | `BoundedRelease` | yes | 7 |
| S06 | the non-final capture budget is not enforced | `CaptureCountWithinProfile` | yes | 6 |
| S07 | closing a hold does not record what had been captured | `NoCaptureAfterClose` | yes | 3 |
| S08 | nothing obliges the provider to ever release the funds | `EventualRelease` | yes | 8 |
| S09 | nothing obliges the provider to ever expire a stale authorization | `EventualClose` | yes | 8 |
| S10 | the hold need never be created | `Termination` | yes | 2 |

## V. FROM MODEL TO CONFORMANCE SUITE

### A. Two models, kept honest

The specification is checked by TLC, but TLC cannot generate test sequences or grade a running service, so the state machine is also implemented in Python. Two models of the same thing is an obvious way to be wrong slowly.

We check it directly: for each profile, both state spaces are enumerated and compared element by element, using TLC's state dump. They are identical for all 6 profiles. This runs as an experiment rather than a one-time check, so any future edit that moves one and not the other fails immediately.

### B. What a test may do and see

A test can call four operations — authorize, capture (with an amount and a final flag), void, increase authorization — and can let time pass. It cannot invoke expiry or the release of funds; those are the provider's to do, and the test only waits and looks.

What it sees is the hold's status, the authorized amount, the captured total, and whether the held funds have been released. The oracle asks the model, before each call, whether a conforming provider must accept it and what must be observable afterwards, then compares. Release is checked against a permitted set rather than a value, since the profile allows it to lag: while the deadline has not passed either answer is conforming, and after it only one is.

### C. Two things that turned out not to be observable

Two fields were in the observation at first and had to come out. Both removals are findings rather than concessions.

*The number of captures is not provider-neutral.* Closing a hold and releasing its remainder costs one capture call on Stripe, where it is a zero-amount capture with `final_capture=true`, and zero on Adyen, where it is a cancel. A suite that counts captures is counting different things on the two rails. This surfaced as a conformance failure in the HTTP run, where the Adyen adapter's translation of one abstract operation into two requests showed up as an extra capture.

*The reason a hold closed is not observable at all, synchronously.* A Stripe PaymentIntent that was voided and one that expired uncaptured both read `canceled` [2]; Adyen reports a cancelled and an expired authorisation alike. Distinguishing them requires the event stream. We removed the field rather than grade providers on something their APIs do not answer.

Both removals cost nothing measurable — detection stayed at 61/61 — but that is evidence about these particular defects, not a proof that nothing needs those fields. A defect that voids a hold where it should have let it expire is invisible to this suite, and consuming the event stream is the way to reach it.

### D. Generating the suite

The suite covers every state a black-box test can drive the provider into, every accepted transition between those states, and, at every state, every call the model says must be refused. Construction is a breadth-first tree over the testable state space: one test walks each root-to-leaf path, and each transition the tree does not use gets one short test. Because the oracle checks after every call, a test that is a prefix of another adds nothing and is dropped. Refusal checks are issued at each state along the way, which is cheap: a refused call leaves the state alone, so they can all be made at one point in a sequence. The criterion is standard for testing from state machines [9], [10]; what is new is the machine it is applied to.

### E. Seeded defects

Twelve mutants, each breaking one rule, in the style of mutation testing [11], [12]: capture after close, capture after expiry, an unbounded ceiling, a ceiling off by one, an extra non-final capture, a double release, no release, a late release, a void accepted after a capture where the profile forbids it, a partial capture that leaves the hold open, an expiry check off by one tick, and a release before the hold closes. Each overrides a single decision in the reference provider, so a mutant differs from a correct implementation by one choice rather than a rewrite.

Not every mutant is detectable. On a single-capture profile, "void accepted after a capture" cannot be reached, because any capture closes the hold first. Rather than assume which are which, we search: a breadth-first walk over the product of mutant and reference under identical call sequences, returning

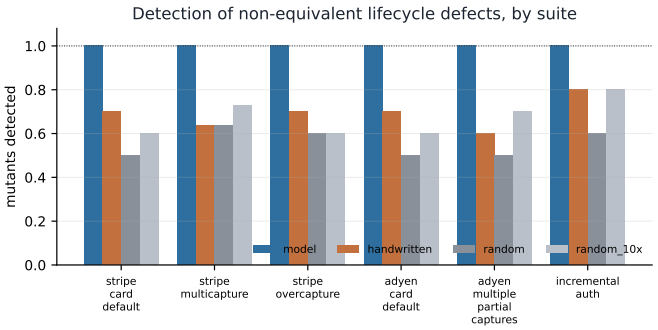


Fig. 2. Detection by suite and profile. The generated suite is at 1.0 everywhere; the baselines vary with which defects happen to lie on a common path.

the shortest distinguishing sequence or, exhaustively within the profile's bounds, nothing. Mutants in the second category are reported as equivalent and excluded from the rates. Of 72 mutant–profile pairs, 11 are equivalent and 61 are killable.

### F. Baselines

Two, both using the same oracle, so what is compared is the choice of call sequences and nothing else. *Random* draws calls uniformly, with the same API-call budget as the generated suite — refusal checks included, since those cost a call too. A second random arm gets ten times that budget, to separate choosing the wrong calls from not making enough of them. *Hand-written* is the suite an integration engineer writes from a provider quickstart: authorize then capture, authorize then void, authorize and let it expire, a partial capture, a capture over the limit, a second capture, a capture after a void, a capture with no authorization, and the profile-specific happy path. Eight to ten tests, no exhaustive refusal checking, which is the point of it.

## VI. Results

### A. Detection

Table IV and Fig. 2 give the outcome. The generated suite detects every killable defect on every profile, 61/61. Equal-budget random reaches 56%, and with ten times the budget 67% — so the gap is in which sequences are chosen, not in how many. The hand-written suite reaches 69% on roughly 25 calls, which is a respectable return for its size and is the honest comparison: it is not a weak baseline, it is a cheap one.

What the baselines miss is consistent and, in hindsight, predictable. They find the defects that break a common path — an unbounded ceiling shows up the first time anything captures too much. They miss the ones that need a specific state reached in a specific order: a capture accepted after a void needs a void first and a capture second, both on a hold that is otherwise fine; a late release needs the clock pushed past a deadline that only exists after the hold closes. The generated suite reaches those because it is built from the state space rather than from a list of scenarios someone thought of.

### B. Black-box over HTTP

An in-process result proves less than it appears to, because the suite and the system under test share a language and a process. We therefore stood up mock providers that speak Stripe's and Adyen's own request shapes over HTTP — `POST /v1/payment_intents/{id}/capture` with `amount_to_capture` and `final_capture` on one, `POST /payments/{ref}/captures` with an amount object on the other — and ran the identical suite through adapters.

Table V reports it. All 4 of 4 deployments conform, and all 12 of 12 injected defects are detected through the network interface.

The value of this run was not the confirmation. It was the failures along the way: the Adyen adapter did not conform at first, and each failure was a real defect in the model rather than in the adapter. That is how the missing final-capture capability was found (§II-C), and how both non-observable fields were found (§V-C). A specification written only against documentation and checked only against itself would have kept all three mistakes.

### C. Where two providers disagree

The same product search that classifies equivalent mutants compares two providers when the two sides are given different profiles. Run to exhaustion it returns every reachable disagreement; divergences reached by different prefixes but making the same attributes differ in the same way are one finding, and the shortest prefix is kept as the witness.

Two adjustments were needed to make the answers mean anything. Both sides get the larger of the two clock bounds, because otherwise the profile with the shorter horizon simply stops advancing and the saturation is reported as a behavioral difference — before this was fixed, three of four reported divergences in the Stripe/Adyen comparison were that artifact. And a class is identified independently of the amount, because a provider that refuses a closing capture refuses it at every amount, and reporting one class per amount reports one disagreement several times. The amounts are kept alongside the class, since for a bound like over-capture the amount at which behavior changes is the finding.

Across all 15 profile pairs, every pair diverges, with up to 5 independent classes (Fig. 3). Table VI gives the three pairs that correspond to real provider comparisons.

*Defaults.* One divergence: the authorization expires a tick earlier on the Stripe profile. This is the documented 7-day versus 10-day validity for Visa card-not-present cardholder-initiated transactions, and the witness is a capture attempt in the window where one provider still holds the funds and the other has released them.

*Over-capture.* Two: a capture of five units against an authorization of four is accepted on the Stripe overcapture profile and refused on Adyen, plus the validity difference. The witness is two calls long.

*Multiple captures.* Four, and these are the ones that matter for anyone building over both. A closing capture below the full authorized amount is accepted on Stripe and refused on Adyen,

TABLE IV

DETECTION OF SEEDED DEFECTS. EQUIVALENT MUTANTS — THOSE NO CALL SEQUENCE CAN DISTINGUISH FROM A CORRECT IMPLEMENTATION, ESTABLISHED BY EXHAUSTIVE SEARCH — ARE EXCLUDED FROM THE RATES. RANDOM ARMS USE THE MODEL SUITE'S CALL BUDGET AND TEN TIMES IT.

| | Mutants | | Model suite | | Fraction detected | | | |
|---|---|---|---|---|---|---|---|---|
| Profile | killable | equiv. | tests | calls | model | random | random×10 | hand |
| Stripe, card default | 10 | 2 | 16 | 2,089 | 1.00 | 0.50 | 0.60 | 0.70 |
| Stripe, multicapture | 11 | 1 | 160 | 16,535 | 1.00 | 0.64 | 0.73 | 0.64 |
| Stripe, overcapture | 10 | 2 | 19 | 2,748 | 1.00 | 0.60 | 0.60 | 0.70 |
| Adyen, card default | 10 | 2 | 21 | 3,072 | 1.00 | 0.50 | 0.60 | 0.70 |
| Adyen, multiple partial captures | 10 | 2 | 151 | 20,094 | 1.00 | 0.50 | 0.70 | 0.60 |
| Incremental auth (synthetic) | 10 | 2 | 553 | 82,506 | 1.00 | 0.60 | 0.80 | 0.80 |
| **All profiles** | **61** | | | | **1.00** | 0.56 | 0.67 | 0.69 |

TABLE V

THE SAME SUITE OVER HTTP, AGAINST SERVICES SPEAKING EACH PROVIDER'S REQUEST SHAPE.

| Profile | Shape | Tests | Calls | Clean | Found |
|---|---|---|---|---|---|
| Stripe, card default | stripe | 16 | 2,089 | conforms | 3/3 |
| Stripe, multicapture | stripe | 160 | 16,535 | conforms | 3/3 |
| Adyen, card default | adyen | 21 | 3,072 | conforms | 3/3 |
| Adyen, multiple partial captures | adyen | 151 | 20,094 | conforms | 3/3 |

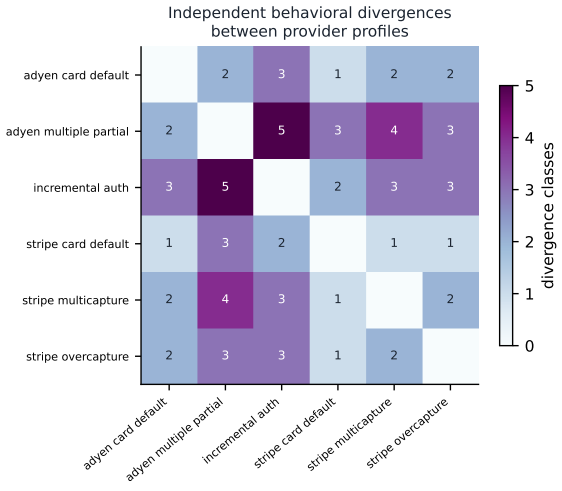


Fig. 3. Independent divergence classes between every pair of profiles. No pair agrees everywhere, including the two Stripe profiles that differ only in an account setting.

because Adyen has no final-capture flag. A non-final capture of the whole authorized amount is accepted on Stripe, which keeps the PaymentIntent open with nothing left to capture, and refused on Adyen, where taking everything ends the hold. A void after a partial capture is refused on Stripe and accepted on Adyen — the same operation, opposite answers. And the validity difference again.

### D. A reproducible defect corpus

Every conformance run and every violation is recorded with the exact call sequence that produced it, in Postgres when one is configured and in SQLite otherwise. The runs reported here left 297 reproducible violations. A violation is thus a script rather than a log line: re-running it reproduces the failure against the same provider profile.

## VII. DISCUSSION

### A. One adapter over two rails is harder than it looks

The usual way to support several PSPs is an internal interface — authorize, capture, void — with an adapter behind each provider. The divergences above say what that interface costs.

Closing a hold short of the full amount is a capture on Stripe and, on Adyen, not expressible as a capture at all. Releasing a remainder is a zero-amount capture on one and a cancel on the other. A void after a partial capture is legal on one and refused on the other. An adapter can hide the first two by composing calls, as ours does, but composition has a visible cost: what was one operation becomes two, and any observable that counts operations stops meaning the same thing. The third cannot be hidden — the operation is simply unavailable on one rail, and the honest adapter refuses it locally rather than sending a request that will be misinterpreted.

The practical reading: a multi-PSP interface should be typed by capability, and the capability vector should be checked, not assumed. That is what a profile is, and checking it is what the suite does.

### B. What the specification is for

The model is small: eleven variables, eight actions, a few thousand states. Its value is not that it is difficult but that it is checkable, and that everything downstream is derived from it rather than restated alongside it. The suite is generated from it, the oracle is it, the provider comparison is it under two parameterizations. When the model was wrong — three times, all found by running the suite against something that did not share its assumptions — one edit fixed the specification, the suite, and the comparison together.

## VIII. LIMITATIONS

*No live provider was tested.* No sandbox credentials were available. Every result concerns behavior the providers document, not behavior they exhibit. Whether Stripe and Adyen

TABLE VI

DIVERGENCES BETWEEN PROVIDER PROFILES, WITH THE SHORTEST CALL SEQUENCE THAT SHOWS EACH ONE. IN THE WITNESSES `AUTH` IS AUTHORIZE, `WAIT` IS ONE TICK OF ELAPSED TIME, `CAP(`$n$`,FIN)` IS A CAPTURE OF $n$ THAT CLOSES THE HOLD AND `CAP(`$n$`,PART)` ONE THAT DOES NOT.

| Call | Left provider | Right provider | Shortest witness |
|---|---|---|---|
| *Stripe, card default* (left) vs. *Adyen, card default* (right) | | | |
| advance_time | status CLOSED | status HELD | `auth(4) ; wait×3` |
| *Stripe, multicapture* (left) vs. *Adyen, multiple partial captures* (right) | | | |
| capture (final), amount 0, 1, 2, 3 | accepted | rejected | `auth(4) ; cap(1,fin)` |
| capture (non-final), amount 1, 2, 3, 4 | accepted | rejected | `auth(4) ; cap(4,part)` |
| void | rejected | accepted | `auth(4) ; cap(1,part) ; void` |
| advance_time | status CLOSED | status HELD | `auth(4) ; wait×3` |
| *Stripe, overcapture* (left) vs. *Adyen, card default* (right) | | | |
| capture (final), amount 5 | accepted | rejected | `auth(4) ; cap(5,fin)` |
| advance_time | status CLOSED | status HELD | `auth(4) ; wait×3` |

honor their own documentation is the question this work was built to ask and has not answered. The adapters that would ask it are released, unrun and marked as such.

*Two obstacles stand between the suite and a live run*, beyond credentials. A real authorization cannot be fast-forwarded, so expiry tests either wait days or are excluded. And Adyen reports modification outcomes by webhook rather than in the response, so a live run needs a notification endpoint; our adapter raises rather than pretending to poll.

*The model is bounded and small.* Four minor units, a handful of ticks, one hold at a time. Defects that depend on magnitude — an overflow, a currency with a different minor-unit scale — are out of reach, as are races between concurrent operations on one authorization, which need the operation identity that the idempotency work supplies.

*"Equivalent" means equivalent within those bounds.* A mutant the search calls indistinguishable might be distinguishable at an amount or a horizon the search does not reach.

*Profiles are one reading of the documentation.* They were built by one author from published pages, quoted and cited so they can be checked, but a misreading would propagate silently into every result that depends on it.

*Detection rates are about these twelve defects.* They are drawn from defects we have seen in practice and they cover each invariant, but a defect class nobody thought of is not in the denominator.

## IX. RELATED WORK

*Formal methods for payments* have concentrated on the cryptographic protocol. Basin et al. [13] built a symbolic Tamarin model of EMV and found two attacks on contactless transactions. That work is about card–terminal authentication at the point of sale; the business logic that governs the resulting authorization afterwards — partial capture, void, expiry — is outside its scope and is the subject here.

*TLA+ applied to payment systems* is closest in method. Grundmann and Hartenstein [14] verify the Lightning Network with a refinement chain to keep the state space tractable, and their practice of reporting state counts and depth rather than asserting a model was checked is one we follow. The domain is different: payment channels, not merchant-side holds.

*Binding specifications to implementations.* Howard et al. [8] validate traces of the Confidential Consortium Framework against a TLA+ specification in CI and find six bugs. This is the model for §V and the reason we treat model-versus-implementation agreement as an experiment rather than an assumption. Their system's source is available, so they can validate traces; ours is a third-party API, so the binding has to be black-box conformance. Newcombe et al. [15] make the broader industrial case for specifying before building.

*Differential testing of APIs.* Godefroid et al. [16] compare versions of a REST service to find breaking changes, driven by the API's interface description. We compare two providers of the same business capability, driven by a semantic state machine, so a divergence comes back as a lifecycle rule rather than a schema change.

*Testing from state machines* is long-settled [9], [10], as is mutation testing [11], [12]. We use both as tools. The contribution is the machine they are applied to and what applying them reveals about the providers.

*Provider documentation* [2]–[6] is careful and, within each provider, complete. None of it is machine-checkable, and none of it describes another provider — which is precisely the gap §VI-C fills.

## X. CONCLUSION

The authorization hold is a piece of financial state that sits between two parties for days, governed by rules that every payment provider implements and none specifies formally. We gave those rules a machine-checked state machine, parameterized it by provider capability, and used it to generate a conformance suite that detects every seeded defect a call sequence can reach — where random testing at ten times the budget reaches two-thirds and a hand-written suite about the same.

The result we did not expect is how much the two providers differ under the same name. Every profile pair diverges. Two of the divergences make the common one-interface-over-many-rails design harder than it is usually treated as being, and one of them — the absence of a final-capture flag — is not a difference in values but in what operations exist.

The obvious next step is the one we could not take: point the suite at a live sandbox and find out whether the providers do what they say. Everything needed to do that is released.

## Artifact Availability

Code, specification, conformance suite, mock providers, sandbox adapters, experiment scripts, and the scripts that regenerate every table and figure are at https://github.com/abhisheksharma2411/holdspec, MIT licensed, tagged `v1.0.0`. `make reproduce` runs the full pipeline from a clean tree: model checking, all six experiments, figures, and this PDF. Total runtime is a few minutes on a laptop; Docker is optional. The archived release is deposited on Zenodo under DOI `10.5281/zenodo.22244465`. There is no separate dataset: provider profiles are version-controlled with a source URL and a verbatim quotation per field, and all workloads are generated from the model at run time.

## References


[1] A. Sharma, "A provider-aware formal contract and conformance suite for payment idempotency," 2026, manuscript under review.

[2] Stripe, "Place a hold on a payment method," https://docs.stripe.com/payments/place-a-hold-on-a-payment-method, 2026, accessed 1 September 2026.

[3] ——, "Capture a payment multiple times," https://docs.stripe.com/payments/multicapture, 2026, accessed 1 September 2026.

[4] ——, "Capture more than the authorized amount on a payment," https://docs.stripe.com/payments/overcapture, 2026, accessed 1 September 2026.

[5] Adyen, "Capture," https://docs.adyen.com/online-payments/capture/, 2026, accessed 1 September 2026.

[6] ——, "Adjust authorisation," https://docs.adyen.com/online-payments/adjust-authorisation/, 2026, accessed 1 September 2026.

[7] M. Abadi and L. Lamport, "An old-fashioned recipe for real time," *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 5, pp. 1543–1571, 1994.

[8] H. Howard, M. A. Kuppe, E. Ashton, A. Chamayou, and N. Crooks, "Smart casual verification of the confidential consortium framework," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 2025, arXiv:2406.17455.

[9] T. S. Chow, "Testing software design modeled by finite-state machines," *IEEE Transactions on Software Engineering*, vol. SE-4, no. 3, pp. 178–187, 1978.

[10] D. Lee and M. Yannakakis, "Principles and methods of testing finite state machines — a survey," *Proceedings of the IEEE*, vol. 84, no. 8, pp. 1090–1123, 1996.

[11] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[12] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[13] D. Basin, R. Sasse, and J. Toro-Pozo, "The EMV standard: Break, fix, verify," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1766–1781, arXiv:2006.08249.

[14] M. Grundmann and H. Hartenstein, "Towards a formal verification of the lightning network with TLA+," *arXiv preprint arXiv:2307.02342*, 2023.

[15] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, "Use of formal methods at Amazon web services," in *Communications of the ACM*, vol. 58, no. 4, 2015, pp. 66–73.

[16] P. Godefroid, D. Lehmann, and M. Polishchuk, "Differential regression testing for REST APIs," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 312–323.